

Experiment 02

Aim: To write metadata in a Web App manifest file for an E-commerce Progressive Web App (PWA) to enable the **"Add to Home Screen"** feature. This functionality allows users to install the web app on their devices and use it like a native application.

Theory:

The purpose of this experiment is to implement a user interface in Flutter by leveraging its collection of commonly used widgets. Flutter is an open-source UI software development toolkit developed by Google, based on the Dart programming language. It follows a declarative and component-based architecture, allowing developers to construct highly customizable and natively compiled interfaces from a single codebase.

In this context, the experiment involves the utilization of fundamental Flutter widgets to structure and compose the UI. Key structural widgets include **Scaffold**, which provides the basic layout structure with built-in support for material design components such as **AppBar**, **FloatingActionButton**, and **Drawer**. The **Column** and **Row** widgets are employed to manage vertical and horizontal layouts respectively, while the **Container** widget is used for styling, alignment, padding, and positioning of child widgets.

Interactive widgets such as **TextFormField** enable user input and form management, often in conjunction with **Form** and **GlobalKey** for validation and state handling. Actionable elements like **ElevatedButton**, **IconButton**, and **GestureDetector** are integrated to provide user interaction and event handling. **ListView** and **GridView** facilitate dynamic list rendering and efficient scrolling behavior.

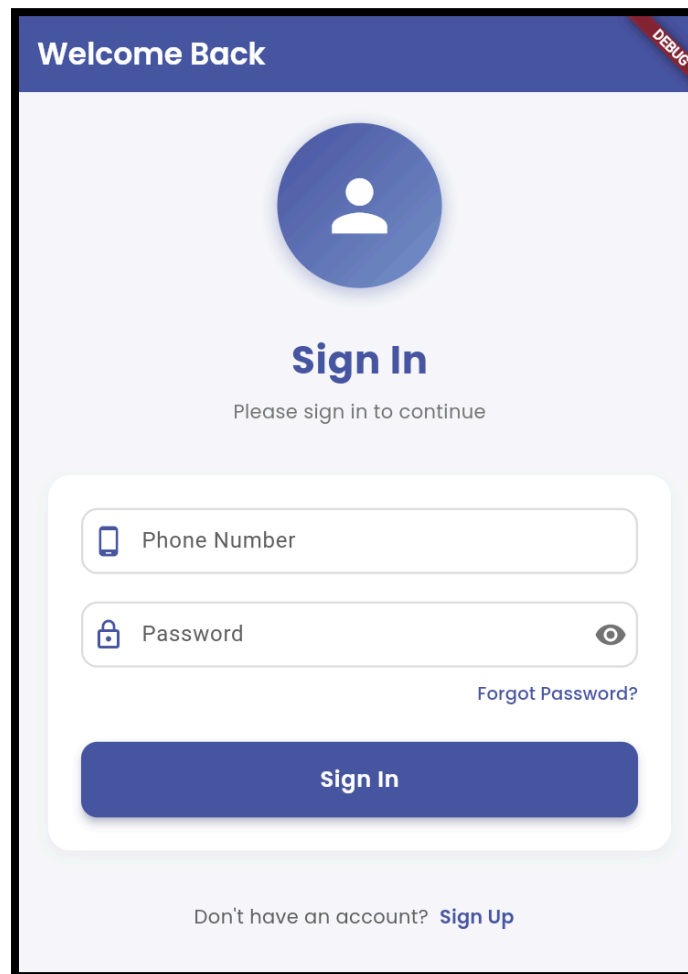
By composing these widgets hierarchically within a widget tree, developers can create modular, reusable, and responsive UIs. This experiment demonstrates the foundational principles of Flutter's widget-based design system and emphasizes the importance of widget composition, state management, and material design adherence in modern cross-platform application development.

Implementation:

A responsive and animated login interface was developed using Flutter, incorporating commonly used widgets and best UI practices. The `LoginScreen` demonstrates structured layout using `Column`, `TextFormField`, `Container`, `Form`, and custom `InputDecoration` for consistent styling. Firebase Firestore is used for user credential verification. Animations such as `FadeTransition` and `Hero` were implemented for visual polish. State management was handled using `StatefulWidget`, `TextEditingController`, and `setState`, with validation applied to ensure correct user input. Navigation between screens is managed with `Navigator.pushReplacement` and animated transitions. The UI is enhanced with gradients, rounded corners, and proper spacing using `SizedBox` and `Padding`, while `SnackBar` provides instant feedback for login outcomes. The login button is dynamically disabled and shows a loading indicator during Firebase operations, demonstrating conditional rendering and user experience considerations.

Github Execution:

https://github.com/AdityaRaorane22/firebase_flutter/blob/main/lib/screens/login_screen.dart

Output:**Conclusion:**

This experiment successfully demonstrated the integration of standard Flutter widgets to build a modern, animated, and user-friendly login UI. It reflects effective state management, validation, Firebase integration, and UI best practices essential for scalable Flutter application development.